



University of Asia Pacific

Department of Computer Science & Engineering

Course Title : Microprocessors and Microcontrollers Lab

Course Code : CSE 316

Report No : 05

Report Title : Water Level and Pump Control

Date : - 05 - 2026

Submitted By :

Name : Md. Osman Goni

Reg. No : 23101012

Section : A1

Submitted To :

Khadiza Sultana Sayma

Lecturer, Department of CSE

University Of Asia Pacific.

1. Experiment Name

Water Level and Pump Control

2. Objective

To measure the water level using an **HC-SR04 ultrasonic sensor** (by calculating distance from the sensor to the water surface) and automatically control a **DC motor/pump** through an **L293D motor driver**. The system stops the pump when the water level is high (short distance) and runs the pump when the water level is low (long distance).

3. Apparatus / Hardware & Software Requirements

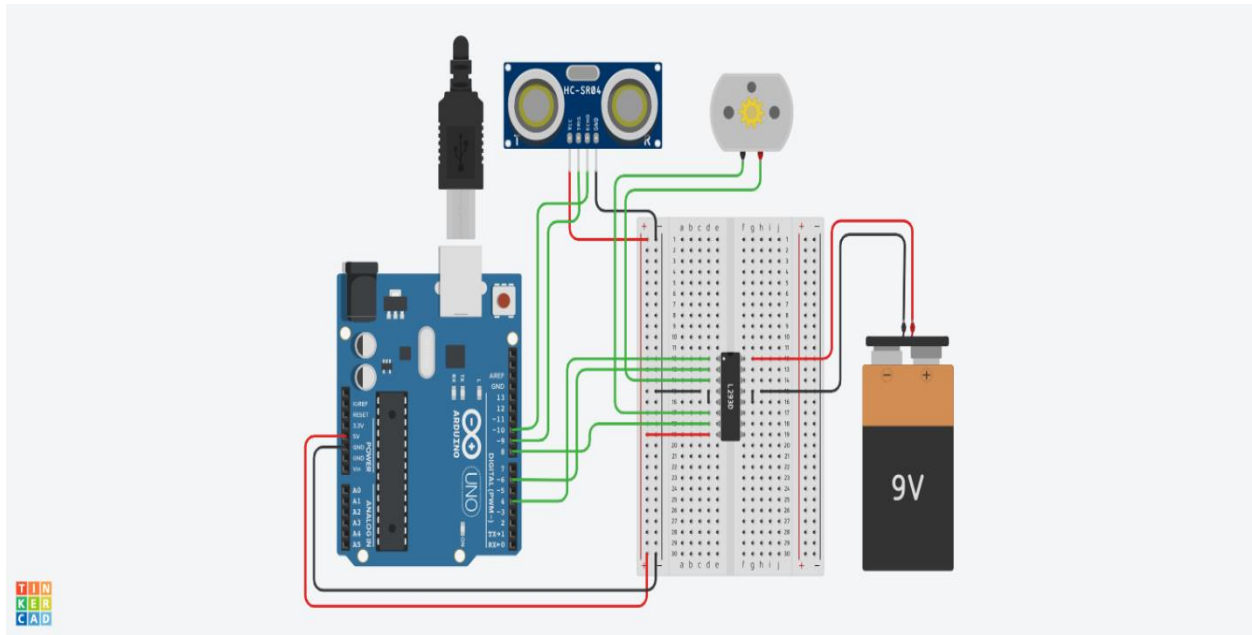
Hardware

- Arduino Uno
- Ultrasonic Sensor (HC-SR04)
- L293D Motor Driver IC
- DC motor
- 9V battery
- Breadboard
- Jumper wires
- (Recommended) common ground between Arduino and external motor supply

Software

- Arduino IDE
- Serial Monitor (for observing distance readings)

4. Circuit Diagram / Schematic



4. Code / Assembly Program

```
// Ultrasonic pins
```

```
const int trigPin = 9;
```

```
const int echoPin = 10;
```

```
// Motor driver pins (L293D)
```

```
const int motorPin1 = 6; // IN1
```

```
const int motorPin2 = 8; // IN2
```

```
const int enablePin = 4; // EN1 (PWM)
```

```
// Variables  
  
long duration;  
  
int distance;  
  
void setup() {  
    pinMode(trigPin, OUTPUT);  
    pinMode(echoPin, INPUT);  
    pinMode(motorPin1, OUTPUT);  
    pinMode(motorPin2, OUTPUT);  
    pinMode(enablePin, OUTPUT);  
    Serial.begin(9600);  
}  
  
void loop() {  
    // Trigger ultrasonic pulse  
    digitalWrite(trigPin, LOW);  
    delayMicroseconds(2);  
    digitalWrite(trigPin, HIGH);  
    delayMicroseconds(10);  
    digitalWrite(trigPin, LOW);  
  
    // Read echo  
    duration = pulseIn(echoPin, HIGH);
```

```
// Convert to distance (cm)
distance = duration * 0.034 / 2;
Serial.print("Distance: ");
Serial.println(distance);

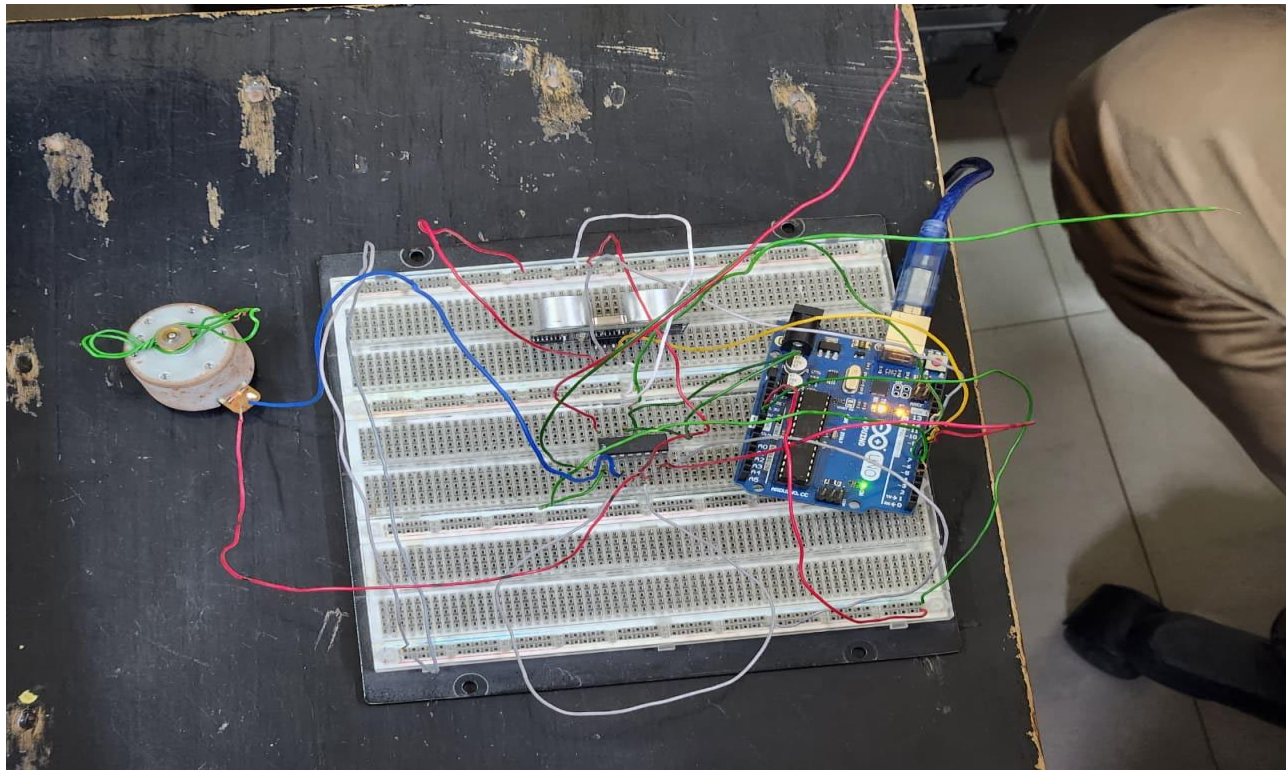
// Control logic
if (distance <10) {
    // Object close → STOP motor
    digitalWrite(motorPin1, LOW);
    digitalWrite(motorPin2, LOW);
    analogWrite(enablePin, 0);
}
else if (distance < 30) {
    // Object close → STOP motor
    digitalWrite(motorPin1, LOW);
    digitalWrite(motorPin2, HIGH);
    analogWrite(enablePin, 200);
}
else {
    // Object far → RUN motor
    digitalWrite(motorPin1, HIGH);
    digitalWrite(motorPin2, LOW);
    analogWrite(enablePin, 200); // speed (0–255)
```

```
}  
delay(200);  
}
```

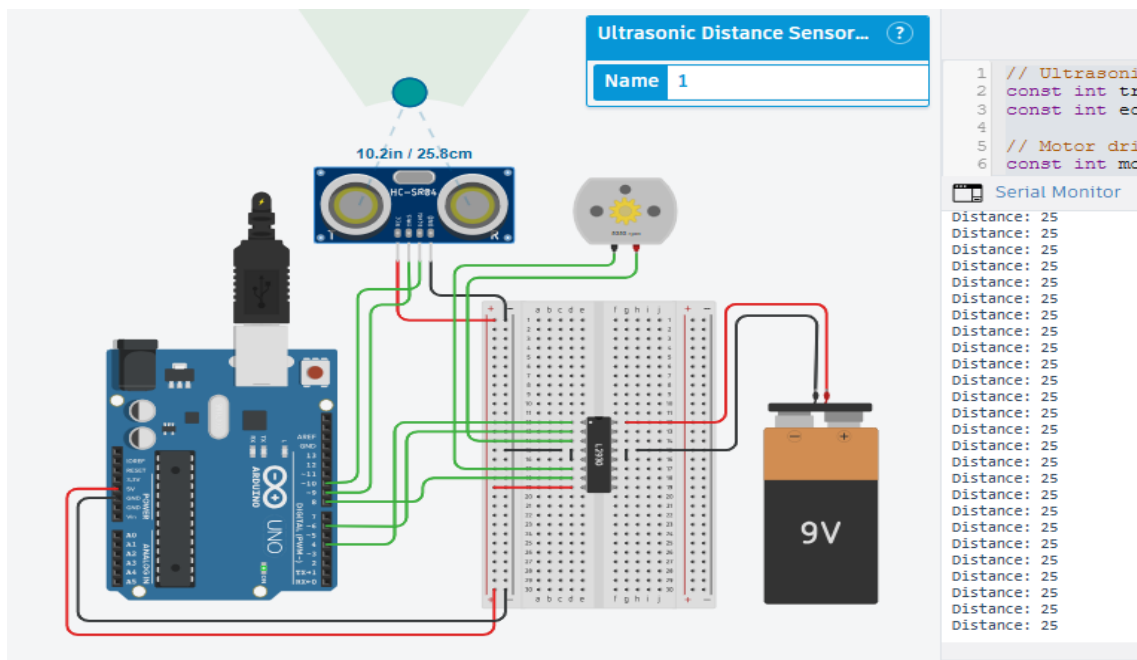
5. Output / Observations

- The ultrasonic sensor continuously sends pulses and measures echo time.
- The distance is calculated in centimeters and printed on the Serial Monitor as: Distance: <value>
- The pump/motor behavior changes automatically depending on the measured distance:
 - **Smaller distance** means water is closer to the sensor (tank more full).
 - **Larger distance** means water is farther from the sensor (tank low).

In Hardware :



In Serial Monitor :



7. Result

After implementing and testing the system, the following observations were recorded:

High Water Level (distance < 10 cm):

When the distance reading was less than 10 cm, the system treated the tank as nearly full. The motor driver outputs were set LOW and the PWM speed was set to 0, so the motor/pump stopped completely, preventing overflow.

Medium Water Level (10 cm to < 30 cm):

When the measured distance was between 10 cm and 30 cm, the motor ran in one direction with PWM ≈ 200 (moderate/high speed). This showed that the system responded correctly in the mid-level range and maintained stable motor operation while continuing to update distance readings.

Low Water Level (distance ≥ 30 cm):

When the distance became 30 cm or more, the system treated the water level as low. The motor ran in the opposite direction with PWM ≈ 200 , indicating the pump /motor activation for low water level condition. The motor continued running until the distance decreased into the higher-level ranges.

8. Conclusion

This experiment provided practical experience in building a sensor-based automatic control system using Arduino. It improved understanding of ultrasonic distance measurement, interpreting distance as a water-level indicator, and controlling a DC motor/pump through the L293D driver using direction pins and PWM speed control. Observing live readings on the Serial Monitor also helped verify correct sensing and decision-making.

Overall, the system successfully changed motor operation based on water level ranges, demonstrating an effective method for automatic tank filling/emptying control. With further refinement—such as setting clear “pump ON/OFF”

thresholds with hysteresis and ensuring a suitable motor power supply the design can be made more stable and reliable for real pump applications.